LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Observation Task No.: 2
Student Name: T. Akshaya
Roll No.: A24126552122
Date:

---

1. TITLE

Student Profile Manager — JavaScript OOP and DOM Manipulation

---

2. AIM

To create a JavaScript application that uses a class and object to store student details, and to dynamically generate and display a student profile on a webpage using DOM manipulation.

---

3. OBJECTIVE

- To define a JavaScript class with properties for storing student details.
- To create an object of the class using values entered by the user.
- To use DOM methods to read input values and create elements dynamically.
- To attach an event listener to a button to trigger profile generation.
- To style the dynamically created profile using CSS.

---

4. THEORY

A class in JavaScript is a blueprint for creating objects that share the same properties. The constructor() method runs automatically when a new object is created using the new keyword, and it is used to set the object's initial property values — in this case, a Student's name, roll number, department, and CGPA.
The DOM (Document Object Model) allows JavaScript to read and change the content of a webpage. Elements are selected using getElementById(), and new elements can be created at runtime using document.createElement(), then added to the page using appendChild(). Properties like textContent set the visible text of an element, and classList.add() applies a CSS class to it. An event listener, added with addEventListener(), runs a function whenever a specific action — such as a button click — occurs, which is how the profile is generated only when the user clicks the button.

---

5. ALGORITHM / PROCEDURE

1. Create a new HTML file and declare <!DOCTYPE html> at the top.
2. Add a <style> section for CSS styling.

3. Create a form container with input fields for Name, Roll Number, Department, and CGPA.
4. Add a "Generate Profile" button.
5. Add an empty to hold the generated profile.
6. In a <script>, define a Student class with a constructor for name, roll, dept, and cgpa.
7. Select the button and the profile container using getElementById().
8. Add a click event listener to the button.
9. Inside the event listener, read and trim the values from the input fields.
10. Validate that all fields are filled before proceeding, and show an alert if any are empty.
11. Create a new Student object using the entered values.
12. Create a new element for the profile card using createElement().
13. Create heading and paragraph elements for each student detail and set their text using textContent.
14. Append all the created elements to the profile card, then append the card to the profile container.
15. Save the file and open it in a browser to test.
16. Test the form validation and profile generation, and record the results as test cases.

---

6. TASK DESCRIPTION

A webpage titled "Student Profile Manager" was created using JavaScript, covering the following:

- Created a Student class with properties: Name, Roll Number, Department, and CGPA.
- Created an object of the Student class using values entered by the user.
- Used the DOM to display the student details dynamically on the webpage.
- Provided a "Generate Profile" button.
- When the button is clicked, the student profile is dynamically created and displayed using DOM manipulation.
- Applied basic CSS styling (card layout, shadow, border-radius, hover effect, and a fade-in animation) to the dynamically generated profile.

---

7. PROGRAM

student_profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Profile Manager</title>
<style>
 body {
   font-family: 'Segoe UI', Arial, sans-serif;
   background: #f0f4f8;
   display: flex;
   flex-direction: column;
   align-items: center;
   padding: 40px;
 }
```

```
.form-container {
  background: #fff;
  padding: 25px 30px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  width: 320px;
}
.form-container h2 {
  text-align: center;
  color: #2c3e50;
}
.form-container input {
  width: 100%;
  padding: 8px;
  margin: 8px 0;
  border: 1px solid #ccc;
  border-radius: 5px;
  box-sizing: border-box;
}
.form-container button {
  width: 100%;
  padding: 10px;
  margin-top: 10px;
  background: #2c7be5;
  color: #fff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 15px;
}
.form-container button:hover {
  background: #1a5cbf;
}
.profile-card {
  margin-top: 25px;
  background: #ffffff;
  border-left: 6px solid #2c7be5;
  border-radius: 8px;
  padding: 20px 25px;
  width: 300px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  animation: fadeIn 0.4s ease-in-out;
}
.profile-card h3 {
  margin-top: 0;
  color: #2c7be5;
}
.profile-card p {
  margin: 6px 0;
  color: #333;
```

```
  }
  @keyframes fadeIn {
    from { opacity: 0; transform: translateY(-10px); }
    to { opacity: 1; transform: translateY(0); }
  }
</style>
</head>
<body>

<div class="form-container">
  <h2>Student Profile Manager</h2>
  <input type="text" id="name" placeholder="Enter Name">
  <input type="text" id="roll" placeholder="Enter Roll Number">
  <input type="text" id="dept" placeholder="Enter Department">
  <input type="text" id="cgpa" placeholder="Enter CGPA">
  <button id="generateBtn">Generate Profile</button>
</div>

<div id="profileContainer"></div>

<script>
  class Student {
    constructor(name, roll, dept, cgpa) {
      this.name = name;
      this.roll = roll;
      this.dept = dept;
      this.cgpa = cgpa;
    }
  }

  const generateBtn = document.getElementById('generateBtn');
  const profileContainer = document.getElementById('profileContainer');

  generateBtn.addEventListener('click', function () {
    const nameVal = document.getElementById('name').value.trim();
    const rollVal = document.getElementById('roll').value.trim();
    const deptVal = document.getElementById('dept').value.trim();
    const cgpaVal = document.getElementById('cgpa').value.trim();

    if (!nameVal || !rollVal || !deptVal || !cgpaVal) {
      alert('Please fill all fields before generating the profile.');
      return;
    }

    const student = new Student(nameVal, rollVal, deptVal, cgpaVal);

    profileContainer.innerHTML = '';

    const card = document.createElement('div');
    card.classList.add('profile-card');
```

```
    const heading = document.createElement('h3');
    heading.textContent = 'Student Profile';

    const nameP = document.createElement('p');
    nameP.textContent = `Name: ${student.name}`;

    const rollP = document.createElement('p');
    rollP.textContent = `Roll Number: ${student.roll}`;

    const deptP = document.createElement('p');
    deptP.textContent = `Department: ${student.dept}`;

    const cgpaP = document.createElement('p');
    cgpaP.textContent = `CGPA: ${student.cgpa}`;

    card.appendChild(heading);
    card.appendChild(nameP);
    card.appendChild(rollP);
    card.appendChild(deptP);
    card.appendChild(cgpaP);

    profileContainer.appendChild(card);
  });
</script>
</body>
</html>
```

---

8. CODE EXPLANATION

Tag / Function	Purpose
class Student {...}	Defines a blueprint (class) for a student, describing the properties every student object will have.
constructor()	Special method that runs when a new object is created; sets the initial values of its properties.
new Student(...)	Creates a new object (instance) of the Student class using the given values.
getElementById()	Selects an HTML element (input field, button, or container) by its id.
addEventListener('click', fn)	Attaches a function to the button that runs whenever it is clicked.
.value.trim()	Reads the text typed into an input field and removes extra spaces.
alert()	Displays a popup message, used here to warn if a field is left empty.
document.createElement()	Creates a new HTML element (like a , <h3>, or <p>) in memory.
classList.add()	Adds a CSS class to an element, used to apply the profile-card style.
textContent	Sets the visible text inside an element, used to display each student detail.

appendChild()	Adds a created element as a child of another element, building up the profile card.
innerHTML = ''	Clears any previously generated profile before creating a new one.
Template literal `${...}`	Inserts a variable's value directly inside a string, used to build each detail line.
<input>, <button>,	Form and container tags used to collect input and hold the generated profile.
@keyframes fadeIn / animation	CSS animation that makes the profile card fade and slide in smoothly when created.

---

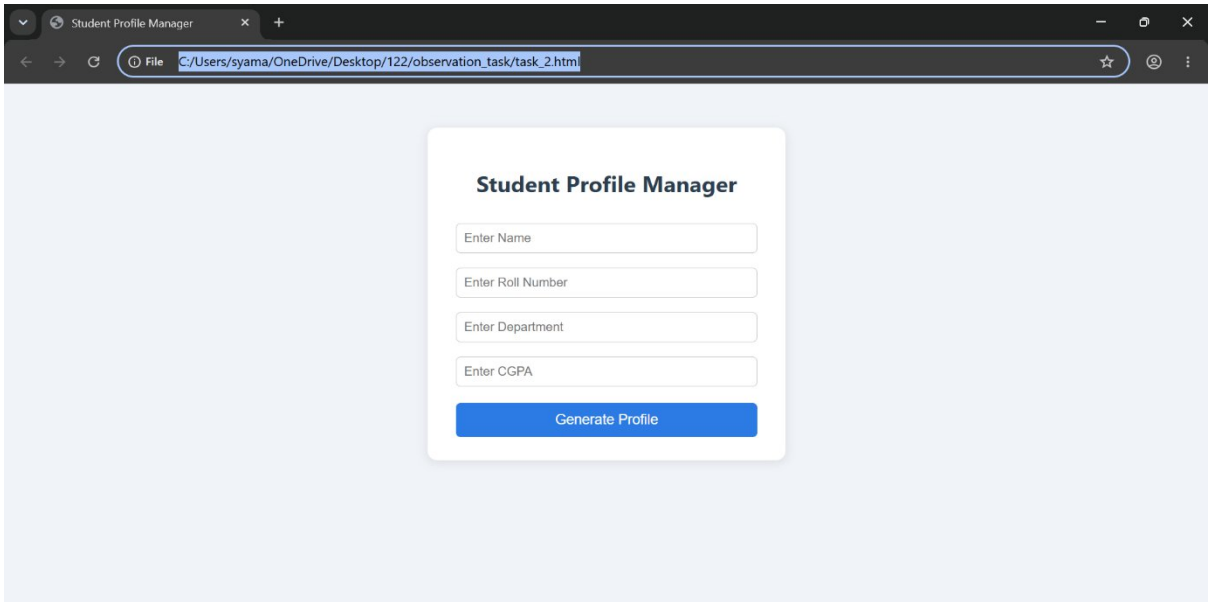
9. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Click "Generate Profile" with empty fields	Alert message is shown, no profile created	Alert shown correctly, no profile displayed	Pass
TC-02	Fill all fields and click "Generate Profile"	Profile card appears with correct student details	Profile card displayed with correct details	Pass
TC-03	Generate a profile again with different values	Old profile is replaced with the new one	Previous card removed, new card displayed	Pass
TC-04	Hover over the "Generate Profile" button	Button background colour changes	Colour changed as expected	Pass
TC-05	Observe the profile card when it is created	Card fades and slides in smoothly	Fade-in animation worked correctly	Pass

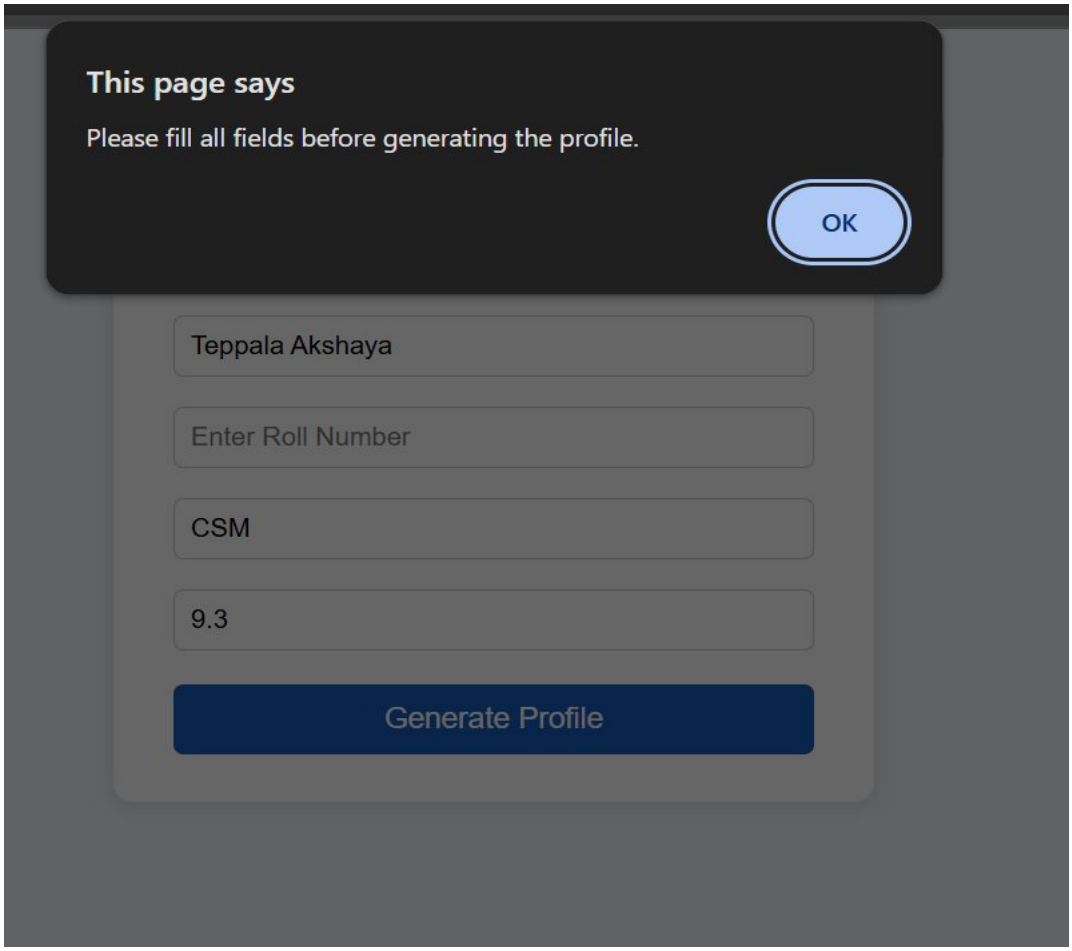
---

10. OUTPUT

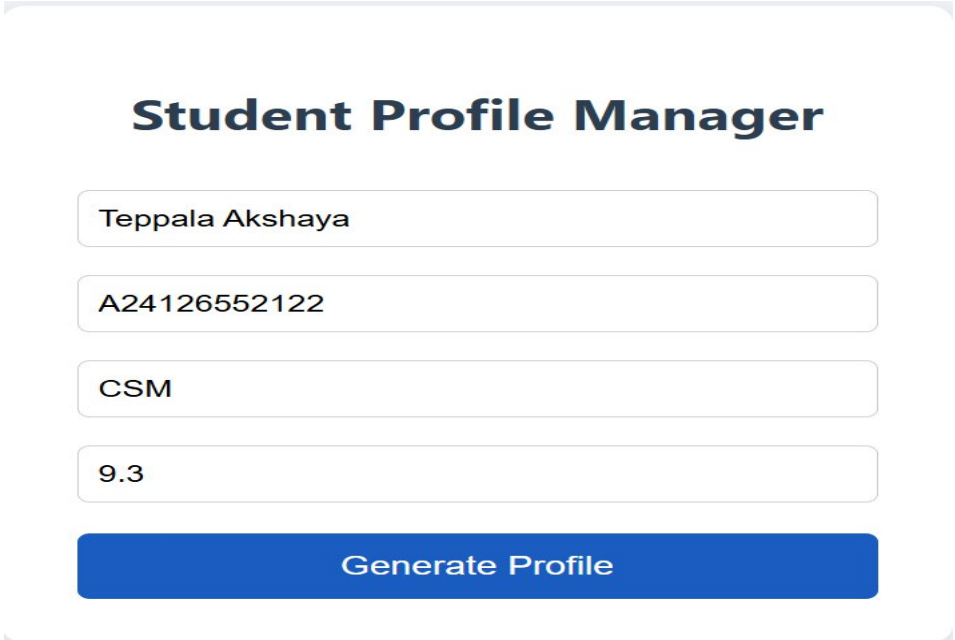
Output 1: Student Profile Manager Form

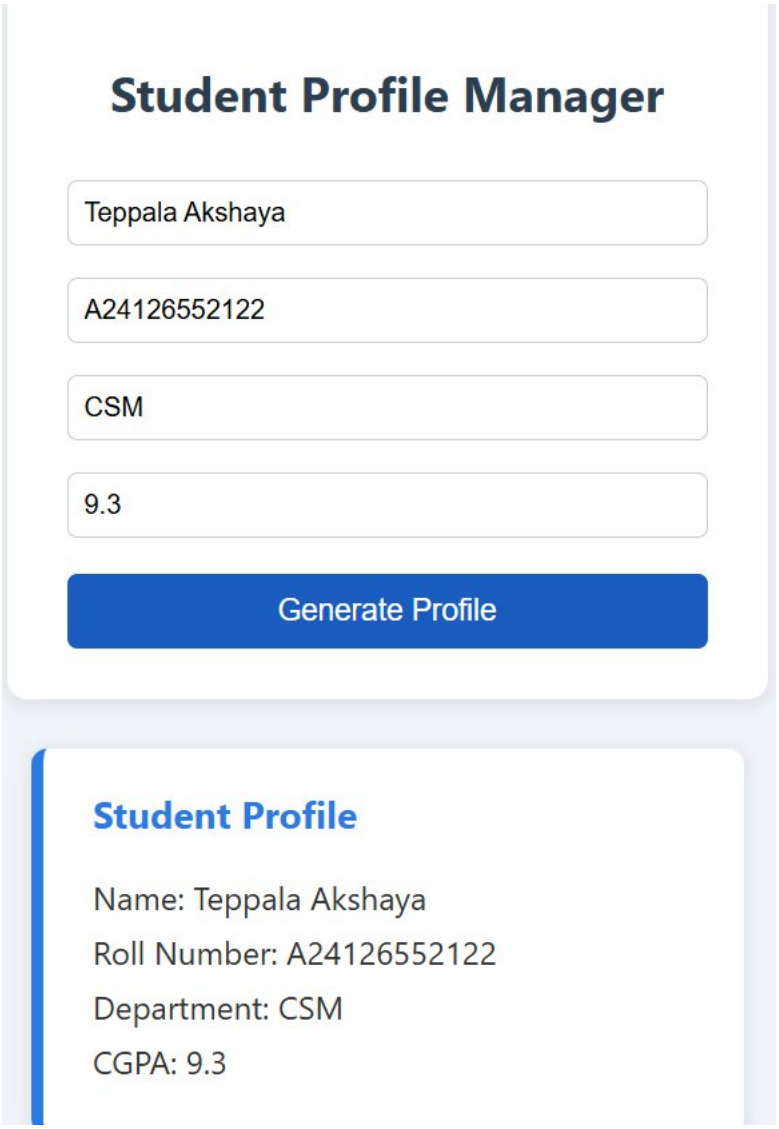


Output 2: Validation Alert (Empty Fields)



Output 3: Generated Student Profile





---

11. RESULT

Thus, a JavaScript application was successfully developed using a Student class and DOM manipulation to dynamically generate and display a student profile on a webpage. The form validation, profile generation, and styling were tested in a web browser and found to work correctly.